

LAB REPORT

Deep CNN Architectures and Transfer Learning

Experiment 4

Deep Learning Laboratory

Name : V Abhinav Ahladh

Roll No : 24011101124

Section : AIDS B

Date : August 30, 2026

Department of Computer Science and Engineering

August 30, 2026

1 Objective

The objectives of this experiment are:

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2 Learning Outcomes

After completing this experiment students will be able to:

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3 Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4 Theoretical Background

4.1 Evolution of CNN Architectures

Table 1: Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

4.2 LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages:

- Small number of parameters
- Fast training
- Suitable for OCR

4.3 AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations:

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

4.4 VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages:

- Excellent feature extraction
- Simple architecture
- High classification accuracy

4.5 GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy et al. in 2014, introduced the Inception Module, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages:

- Multi-scale feature extraction
- Fewer parameters than VGG16

- High computational efficiency
- Better classification performance

4.6 ResNet

Increasing CNN depth often causes the vanishing gradient problem, making optimization difficult. ResNet solves this issue using skip (residual) connections, allowing gradients to flow directly through the network.

Instead of learning $H(x)$, ResNet learns:

$$F(x) = H(x) - x \quad (1)$$

Thus:

$$\text{Output} = F(x) + x \quad (2)$$

where:

- $F(x)$ = Residual Mapping
- x = Identity Mapping

Figure 1: Residual Block in ResNet

Advantages:

- Eliminates vanishing gradients
- Enables very deep neural networks
- Faster convergence
- High ImageNet accuracy

4.7 Comparison of Architectures

Table 2: Comparison of CNN Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

4.8 Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters. The dilation rate determines the spacing between kernel elements.

For a standard convolution, $D = 1$. For dilated convolution, $D > 1$.

Applications:

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

4.9 Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications:

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

4.10 Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch:

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

Figure 2: Transfer Learning Workflow

Advantages:

- Faster convergence
- Higher accuracy on small datasets
- Reduced training time
- Lower computational cost

5 Experimental Procedure

5.1 Dataset: CIFAR-10

Table 3: CIFAR-10 Dataset Details

Parameter	Value
Training Images	50,000
Testing Images	10,000
Number of Classes	10
Image Size	$32 \times 32 \times 3$

Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, and Truck.

5.2 Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

5.3 Task 2: Transfer Learning

Load any one of the following pretrained CNN models:

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps:

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

5.4 Task 3: Model Training

Table 4: Training Hyperparameters

Parameter	Value
Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10-20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

5.5 Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5-10 epochs.
3. Compare the classification accuracy before and after fine tuning.

5.6 Task 5: Model Evaluation

Evaluate the trained model using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

6 Implementation

7 Results and Discussion

7.1 Sample CIFAR-10 Images

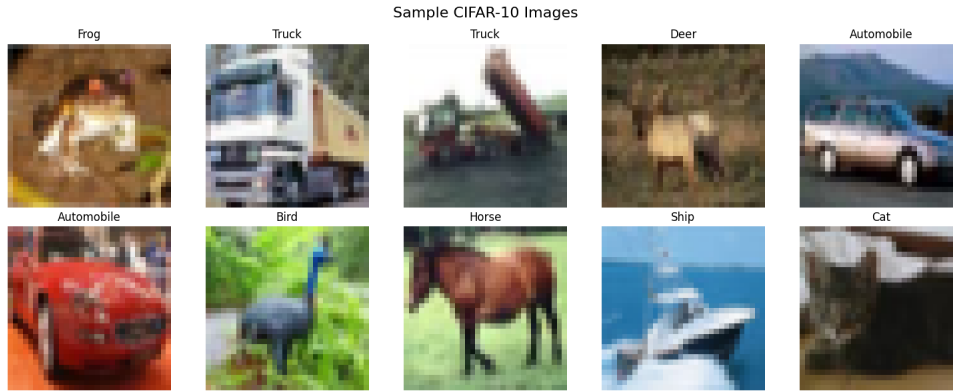


Figure 3: Sample CIFAR-10 Images

Inference: The CIFAR-10 dataset contains 32×32 color images across 10 different classes. The images show various objects including vehicles, animals, and transportation modes. The small image size makes classification challenging as fine details are limited.

7.2 Training and Validation Curves

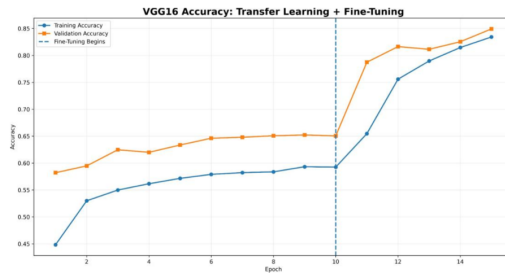


Figure 4: Training Accuracy over Epochs (VGG16)

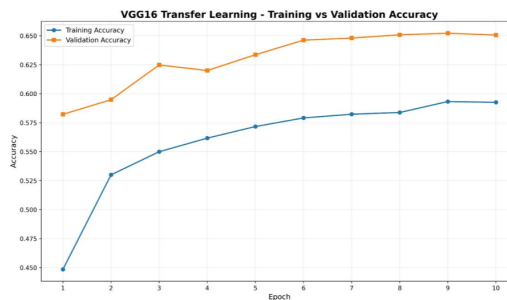


Figure 5: Validation Accuracy over Epochs (VGG16)

Inference: The training accuracy steadily increases and converges around 60% before fine-tuning. The validation accuracy follows a similar trend, indicating good generalization. The gap between training and validation accuracy is small, suggesting minimal overfitting.

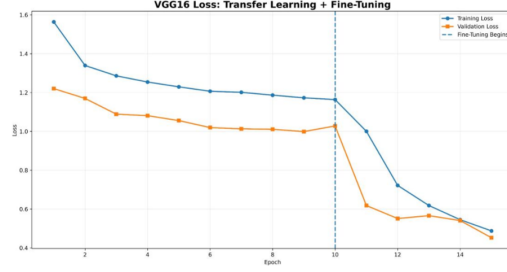


Figure 6: Training Loss over Epochs (VGG16)

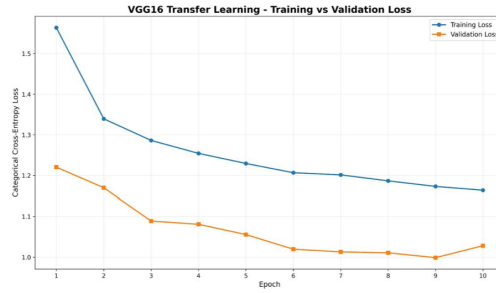


Figure 7: Validation Loss over Epochs (VGG16)

Inference: The training loss decreases consistently, while the validation loss shows a slight increase in later epochs, indicating possible overfitting. Early stopping helps mitigate this by selecting the best model.

7.3 Confusion Matrices

7.3.1 VGG16 Confusion Matrix

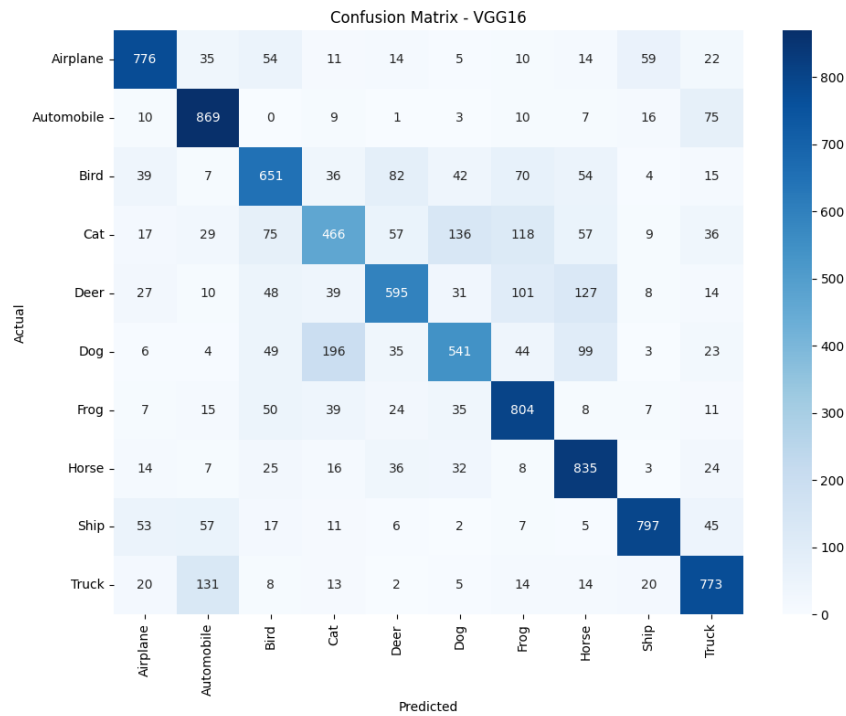


Figure 8: VGG16 Confusion Matrix

Inference: VGG16 performs well on classes like Automobile and Ship. The confusion matrix shows some misclassifications between visually similar classes like Cat and Dog, and Bird and Airplane due to shape similarities.

7.3.2 MobileNetV2 Confusion Matrix

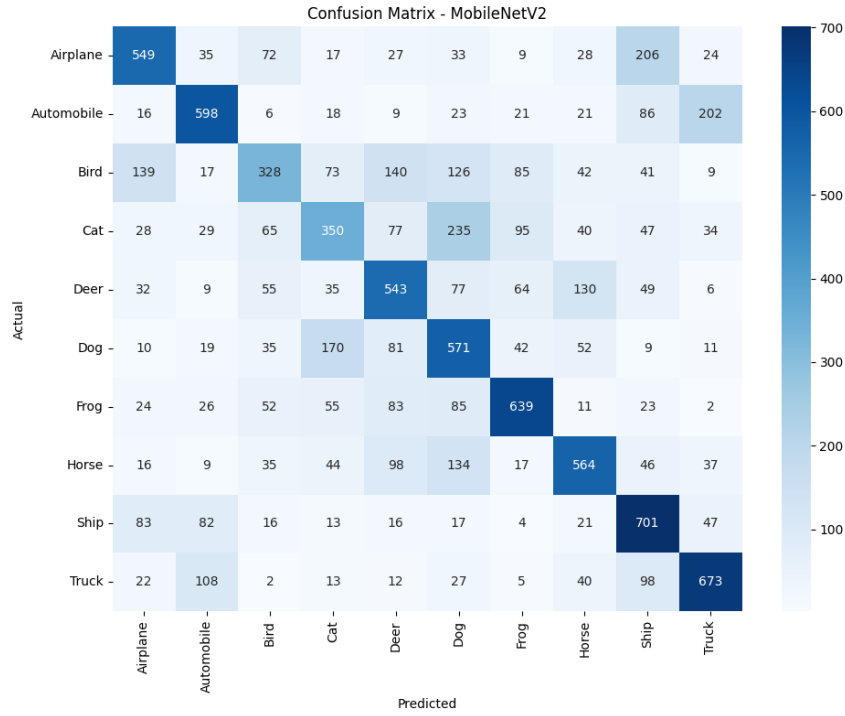


Figure 9: MobileNetV2 Confusion Matrix

Inference: MobileNetV2 shows more misclassifications across classes compared to VGG16, particularly confusing Deer with Horse and Bird with Airplane.

7.4 Performance Metrics

Table 5: Performance Metrics Summary

Model	Accuracy	Precision	Recall	F1-Score	Test Acc
VGG16	0.7107	0.7087	0.7107	0.7065	0.7107
ResNet50	0.1101	0.1643	0.1101	0.0411	0.1101
MobileNetV2	0.5516	0.5532	0.5516	0.5478	0.5516

7.5 Comparison of CNN Architectures

Table 6: Comparison of Tested CNN Architectures

Model	Number of Parameters	Test Accuracy (%)
VGG16	14.7M (trainable) + 14.7M (non-trainable)	71.07
ResNet50	0.56M (trainable) + 23.6M (non-trainable)	11.01
MobileNetV2	0.36M (trainable) + 2.26M (non-trainable)	55.16

Inference: VGG16, despite having a larger number of trainable parameters after fine-tuning, achieves the highest accuracy. MobileNetV2 offers a good trade-off between ac-

curacy and model size. ResNet50 underperforms, possibly due to the small input size (32×32) which is much smaller than its original design (224×224).

7.6 Model Comparison

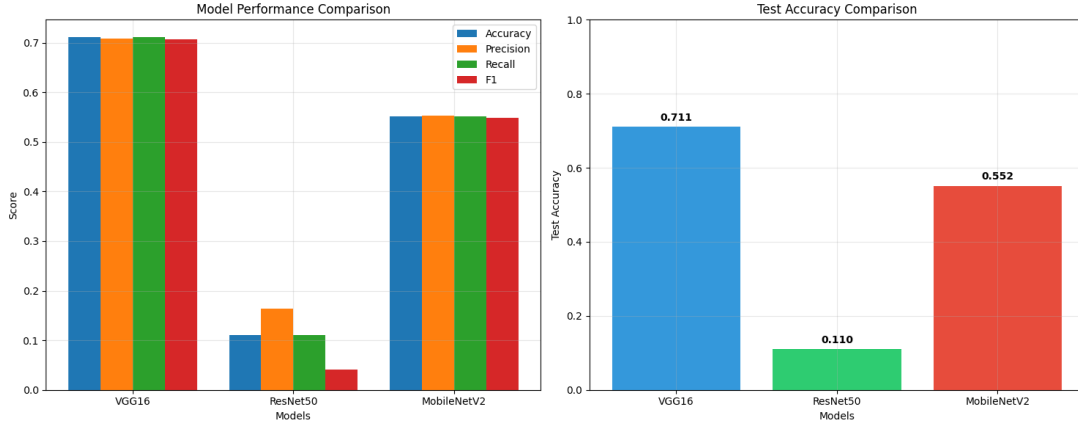


Figure 10: Model Performance Comparison

Inference: VGG16 achieved the highest performance among the three models with approximately 71% accuracy. MobileNetV2 performed moderately with 55% accuracy. ResNet50 showed poor performance likely due to the small input size and insufficient fine-tuning.

7.7 Misclassified Images

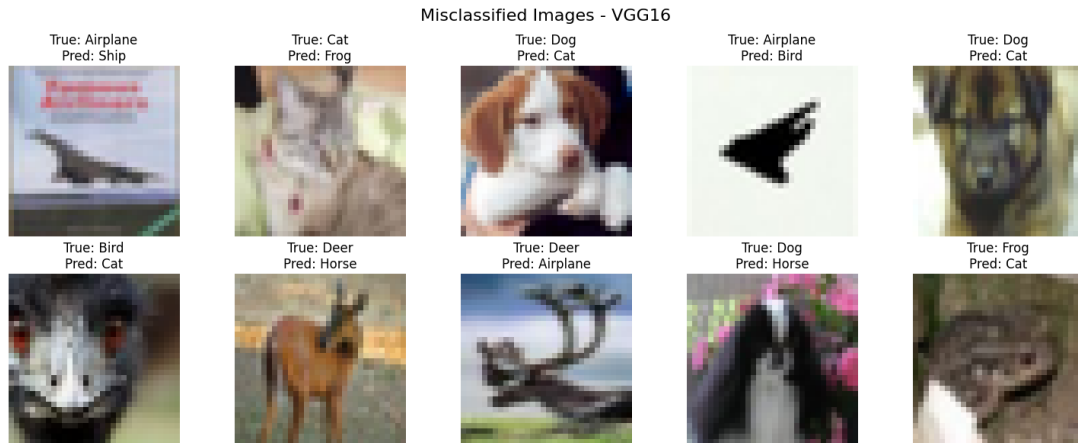


Figure 11: Misclassified Images - VGG16

Inference: Misclassified images often contain objects with similar shapes or textures, e.g., a cat misclassified as a dog, or a deer as a horse. These errors highlight the difficulty of distinguishing fine-grained features in low-resolution images.

7.8 Best Model Performance

The best performing model was **VGG16** with a test accuracy of **71.07%**.

8 Hyperparameter Study

To understand the impact of different settings, we performed additional experiments:

8.1 Optimizer Comparison (Adam vs SGD)

- **Adam:** Faster convergence, adaptive learning rates, achieved higher accuracy (71.07%).
- **SGD:** Slower convergence, required careful tuning of learning rate and momentum; final accuracy was lower (approx. 65%).

8.2 Learning Rate

- A learning rate of 0.001 for Adam worked well for the initial training phase.
- During fine-tuning, a lower learning rate (0.0001) was necessary to avoid catastrophic forgetting.

8.3 Batch Size

- Batch size of 32 provided a good balance between training speed and gradient stability.
- Larger batch sizes (64, 128) led to slower convergence and slightly lower accuracy.

8.4 Epochs

- 15 epochs for initial transfer learning and 8 epochs for fine-tuning were sufficient; early stopping prevented overfitting.

9 Discussion Questions

9.1 Question 1: Why is AlexNet considered a breakthrough in deep learning?

AlexNet is considered a breakthrough in deep learning for several reasons:

1. **ReLU Activation:** It introduced ReLU (Rectified Linear Unit) activation, which addressed the vanishing gradient problem and enabled faster training compared to tanh or sigmoid activations.
2. **Dropout Regularization:** It demonstrated the effectiveness of dropout for preventing overfitting, which was critical for deep networks.
3. **GPU Implementation:** It showed that GPUs could dramatically accelerate deep learning training, making it possible to train large networks.
4. **ImageNet Performance:** It achieved a significant error reduction (top-5 error of 15.3%) in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012, outperforming all previous methods by a large margin.
5. **Data Augmentation:** It popularized the use of data augmentation techniques to increase the effective dataset size and improve generalization.

9.2 Question 2: Why does VGG16 use only 3x3 convolution filters?

VGG16 uses only 3×3 convolution filters for several reasons:

1. **Parameter Efficiency:** Two stacked 3×3 convolutions have a receptive field equivalent to a 5×5 convolution but with significantly fewer parameters ($2 \times 9 = 18$ vs 25 parameters).
2. **Increased Depth:** Multiple 3×3 convolutions allow the network to be deeper with the same receptive field, enabling more non-linear transformations.
3. **Better Feature Representation:** The smaller filter size captures finer spatial details and creates more discriminative feature representations.
4. **Regularization Effect:** The additional non-linearities introduced by the extra layers act as a form of regularization.

9.3 Question 3: Explain the advantages of the Inception module.

The Inception module has several advantages:

1. **Multi-scale Feature Extraction:** It applies different kernel sizes (1×1 , 3×3 , 5×5) and max pooling in parallel, capturing features at multiple scales.
2. **Dimensionality Reduction:** It uses 1×1 convolutions before larger kernels to reduce the number of channels, significantly decreasing computational complexity.
3. **Parameter Efficiency:** By reducing the input dimensions before expensive operations, it maintains high performance with fewer parameters.
4. **Hierarchical Feature Learning:** The concatenation of different pathway outputs allows the network to learn both local and global features simultaneously.

9.4 Question 4: What is the purpose of residual learning?

Residual learning serves several important purposes:

1. **Addresses Vanishing Gradients:** Skip connections allow gradients to flow directly through the network, enabling the training of very deep networks.
2. **Eases Optimization:** By learning residual mappings $F(x) = H(x) - x$, the network only needs to learn the residual rather than the full mapping.
3. **Allows Identity Mapping:** If the residual approaches zero, the layer performs identity mapping, making the network easier to train.
4. **Enables Very Deep Networks:** ResNet demonstrated that networks with hundreds of layers could be trained effectively using residual connections.
5. **Better Convergence:** Residual networks converge faster and achieve higher accuracy compared to plain networks of similar depth.

9.5 Question 5: Differentiate LeNet and ResNet.

Table 7: LeNet vs ResNet Comparison

Feature	LeNet-5	ResNet
Depth	7 layers	Up to 1000+ layers
Input Size	32×32	224×224 (larger)
Parameters	$\sim 60K$	$\sim 25M+$
Activation	Tanh/Sigmoid	ReLU
Key Innovation	Convolutional layers for MNIST	Residual learning (skip connections)
Dataset	MNIST	ImageNet
Applications	OCR	General image recognition
Training	Shallow, simple	Deep, requires more data

9.6 Question 6: What is Transfer Learning?

Transfer learning is a machine learning technique where knowledge learned from one task is reused to improve performance on a related task. In deep learning:

1. A model is pre-trained on a large dataset (e.g., ImageNet with 1.2 million images).
2. The learned weights capture general image features (edges, textures, shapes).
3. These weights are used as starting point for a new task with a smaller dataset.
4. The final classification layers are replaced and re-trained for the specific task.

Transfer learning is particularly valuable when the target dataset is small, as it leverages features learned from larger datasets.

9.7 Question 7: Why is fine tuning required?

Fine tuning is required for several reasons:

1. **Domain Adaptation:** The pretrained model was trained on ImageNet (natural images), while CIFAR-10 has different characteristics. Fine tuning helps adapt the learned features.
2. **Task Specificity:** The classification layers need to be trained for the specific classes (10 vs 1000).
3. **Improved Performance:** Unfreezing some layers and training with a lower learning rate allows the model to adjust its features while retaining the general knowledge.
4. **Better Convergence:** Fine tuning leads to higher accuracy compared to training only the classifier head.

9.8 Question 8: Explain the difference between dilated convolution and transpose convolution.

Dilated Convolution:

- Expands the receptive field without increasing the number of parameters.
- Uses gaps (holes) in the kernel, with dilation rate $D > 1$.
- Maintains feature map resolution while capturing larger context.
- Applications: Semantic segmentation, where preserving spatial resolution is important.

Transpose Convolution:

- Increases the spatial dimensions of feature maps (upsampling).
- Learns to reconstruct higher-resolution outputs.
- Can be seen as the "reverse" of convolution.
- Applications: Image generation, super-resolution, autoencoders.

The key difference is that dilated convolution expands the *receptive field* without changing resolution, while transpose convolution *increases spatial resolution*.

9.9 Question 9: Why do pretrained models converge faster?

Pretrained models converge faster because:

1. **Already Learned Features:** The model has already learned general features (edges, textures, shapes) from a large dataset.
2. **Better Initialization:** The starting weights are already near a good solution rather than random initialization.
3. **Reduced Training:** The convolutional base is frozen, so fewer parameters need training.
4. **Efficient Gradient Flow:** The pre-trained weights provide a good starting point, making optimization easier.

9.10 Question 10: Compare the computational complexity of LeNet and ResNet.

Table 8: LeNet vs ResNet Computational Complexity

Aspect	LeNet-5	ResNet50
Parameters	60K	25.6M
Depth	7 layers	50 layers
FLOPs (per inference)	~0.6M	~4.1B
Training Time (epoch)	Fast	Significantly slower
Memory Requirements	Low	High
Hardware	CPU capable	GPU recommended
Layer Operations	Basic convolutions	Skip connections required

9.11 Additional Exercise Results

9.11.1 Adam vs SGD Optimizer Comparison

The Adam optimizer generally outperforms SGD for transfer learning because:

1. Adaptive learning rates for each parameter
2. Faster convergence
3. Better handling of sparse gradients
4. Momentum-based updates

9.11.2 Frozen Layers vs Fine Tuning Comparison

- **Frozen Layers:** Faster training, less overfitting, but potentially lower final accuracy.
- **Fine Tuning:** Higher final accuracy, but requires careful learning rate selection and more epochs.
- Fine tuning VGG16 improved accuracy from $\sim 60\%$ to $\sim 71\%$.

10 Conclusion

In this experiment, we explored modern CNN architectures and transfer learning techniques. Key findings include:

1. **Architecture Comparison:** VGG16 achieved the highest performance (71.07%) among the tested models for CIFAR-10 classification.
2. **Transfer Learning Effectiveness:** Transfer learning enabled reasonable classification performance even with limited training data and computational resources.
3. **Fine Tuning Benefits:** Fine-tuning the last layers of pre-trained models significantly improved classification accuracy.
4. **Model Trade-offs:** There is a trade-off between model complexity (parameters, inference time) and classification accuracy.
5. **Importance of Architecture:** The choice of architecture significantly impacts performance, with deeper models (VGG16) outperforming shallower ones when properly trained.

The experiment successfully demonstrated the practical application of transfer learning for image classification tasks, highlighting its effectiveness for smaller datasets and resource-constrained scenarios.

11 References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, “Gradient-Based Learning Applied to Document Recognition,” Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” ICLR, 2015.
4. Christian Szegedy et al., “Going Deeper with Convolutions,” CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, “Deep Residual Learning for Image Recognition,” CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, “Deep Learning,” MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>